TEMA 6b.- CONSULTAS BÁSICAS

Contenido

1 Consulta de registros. La sentencia SELECT 2
2 Las cláusulas SELECT y FROM 2
3 La cláusula WHERE 3
4 La cláusula ORDER BY 3
5 Alias de columnas 3
6 Operadores 4
6.1 Operadores aritméticos 4
6.2 Operadores de comparación 4
6.3 Operadores lógicos. 4
6.3.1 Operador AND. 4
6.3.2 Operador OR. 5
6.3.3 Operador NOT. 5
6.4 Operador de comparación de cadenas de caracteres (LIKE) 5
6.5 Operadores IS NULL y IS NOT NULL 6
6.6 Operador IN 6
6.7 Operador BETWEEN 7
6.8 Operadores de concatenación 7
7 Cláusula TOP. 8
8 Funciones agregadas o de resumen. 8
8.1 Función COUNT(). 8
8.2 Función SUM(). 8
8.3 Función MIN() 8
8.4 Función MAX() 9
8.5 Función AVG() 9
9 Funciones escalares 9
9.1 Función UPPER() 9
9.2 Función LOWER() 9
9.3 Función DATALENGTH() o LEN() 9
9.4 Función CONCAT() 9
9.5 Función TRIM() 9
9.6 Función REPLACE() 10
9.7 Función REVERSE() 10
9.8 Función SYSDATETIME() 10
9.9 Función ROUND() 10
9.10 Funciones de tipo fecha 10
10 Cláusula GROUP BY 10
11 Cláusula HAVING 11
12 Consultas multitabla 12

TEMA 6b.- CONSULTAS BÁSICAS

1 Consulta de registros. La sentencia SELECT

La consulta de registros es la operación más compleja, y también la más ejecutada, de una Base de Datos. Se lleva a cabo con la sentencia SELECT, que tiene el siguiente formato:

```
SELECT [ALL / DISTINCT]  [expre_colum1, expre_colum2, .., expre_column | * ]

FROM  [nombre_tabla1, nombre_tabla2, ....., nombre_tablan]

[WHERE condición ]

[ORDER BY expre_colum [DESC I ASC] [, expre_colum [DESC I ASC]...];
```

Donde ***expre_colum*** puede ser una columna de una tabla, una constante, una expresión aritmética, una función o varias funciones anidadas.

Veamos para que sirve cada una de las cláusulas de esta sentencia:

2 Las cláusulas SELECT y FROM

La cláusula SELECT se utiliza para selecciona las columnas que se quieren visualizar como resultado de la consulta. Se puede seleccionar cualquier columna de las tablas afectadas por la consulta (cláusula FROM), valores constantes establecidos a la hora de ejecutar la consulta, o bien el comodín '*' para indicar que se quieren visualizar todas las columnas afectadas.

La cláusula FROM permite indicar con qué tablas se trabajará en la consulta. No siempre serán tablas de las que se visualicen columnas, puesto que muchas veces sólo se utilizarán para relacionar unas tablas con otras. En cualquier caso, se usen para que se visualicen sus campos o bien para relacionar otras tablas (que no están directamente relacionadas), se deben indicar en esta cláusula.

Veamos algunos ejemplos:

- Consulta de una columna de una tabla

```
-- Nombre de todos los usuarios (incluye repeticiones)

SELECT nombre

FROM usuarios;

-- Nombre de todos los usuarios (no incluye repeticiones)

SELECT DISTINCT nombre

FROM usuarios;
```

- Consulta de dos columnas de una tabla

```
-- Nombre y apellidos de todos los usuarios

SELECT nombre, apellidos

FROM usuarios;
```

- Consulta de todas las columnas de una tabla

```
-- Toda la información de todos los usuarios

SELECT *

FROM usuarios;
```

3 La cláusula WHERE

La cláusula WHERE permite establecer condiciones sobre que filas se mostrarán en una sentencia de consulta. En ausencia de esta cláusula se muestran todos los registros de la tabla (aunque sólo las columnas establecidas en la cláusula SELECT). Si se indican condiciones mediante la cláusula WHERE sólo se mostrarán aquellas filas que las cumplan.

Veamos algunos ejemplos:

```
-- Nombre y dirección de los polideportivos de Zaragoza

SELECT nombre, direccion

FROM polideportivos

WHERE ciudad = 'Zaragoza';
```

4 La cláusula ORDER BY

La cláusula ORDER BY permite ordenar el resultado de cualquier consulta atendiendo al campo o campos especificados en esta cláusula, ya sea en orden ascendente (ASC) o descendente (DESC).

Por defecto, la ordenación es ascendente.

Ejemplos:

```
-- Nombre y apellidos de los usuarios, ordenados por nombre

SELECT nombre, apellidos

FROM usuarios

ORDER BY nombre ASC;

-- Nombre y apellidos de los usuarios, ordenados por apellidos
descendentemente y nombre ascendentemente

SELECT nombre, apellidos

FROM usuarios

ORDER BY apellidos DESC, nombre ASC;
```

5 Alias de columnas

A veces el nombre de una columna resulta demasiado largo, si resulta demasiado largo podemos utilizar un alias de columna. Éste se pone entre comillas dobles a la derecha de la columna.

SELECT campo1 “alias_campo1”,…

SELECT campo1 as “alias_campo1”,…

Ejemplo:

```
-- Usando un alias para los campos nombre y apellidos

SELECT nombre “nom”, apellidos “apell”

FROM usuarios

ORDER BY nombre ASC;
```

6 Operadores

6.1 Operadores aritméticos

Sirven para formar expresiones con constantes, valores de columnas y funciones de valores de columnas. Devuelven un valor numérico como resultado de realizar los cálculos indicados.

Operador	***Significado***
+	*Suma*
-	*Resta*
***	*Multiplicación*
/	*División*
%	*Módulo* *Devuelve el resto entero de una división. Por ejemplo, 12 % 7 = 5 porque el resto de 12 dividido entre 7 es 5.*

Ejemplo:

```
--Creamos una columna calculada con el alias precio_iva
SELECT id,codigo, tipo, precio, 1.21 * precio "precio_iva"
FROM pistas;
```

6.2 Operadores de comparación

- Igual (=)
- Distinto (!=)
- Menor que (<)
- Mayor que (>)
- Menor o igual (<=)
- Mayor o igual (>=)

Ejemplo:

```
-- Muestra todos los usuarios con un descuento distinto de 0.1
SELECT *
FROM usuarios
WHERE descuento != 0.1;
```

6.3 Operadores lógicos.

6.3.1 Operador AND.

Se utiliza cuando queremos que se cumplan las dos condiciones a la vez. La condición resultante será cierta, solo si ambas son ciertas.

```
-- Muestra aquellos usuarios con un id mayor a 10 y que sean de la ciudad de
Zaragoza
SELECT *
FROM usuarios
WHERE id > 10 AND ciudad = 'Zaragoza';
```

6.3.2 Operador OR.

Se utiliza cuando queramos que se cumpla la primera condición, o la segunda o ambas). La condición resultante será cierta si alguna de las dos lo es.

```
-- Muestra aquellos usuarios que sean de la ciudad de Zaragoza o de Huesca
SELECT *
FROM usuarios
WHERE ciudad = 'Zaragoza' OR ciudad = 'Huesca';
```

6.3.3 Operador NOT.

Se utiliza cuando no queremos que se cumpla una condición.

```
-- Muestra aquellos usuarios que no sean de la ciudad de Zaragoza
SELECT *
FROM usuarios
WHERE NOT (ciudad = 'Zaragoza');
```

Combinación de operadores AND y OR

Estos operadores se pueden combinar de forma ilimitada pero se deben utilizar paréntesis para agrupar *aquellas* expresiones que deseamos que se evalúen juntas.

*SELECT * FROM empleados WHERE salario > 2000 AND (depart=10 OR depart=20);*

No es igual a:

*SELECT * FROM empleados WHERE salario > 2000 AND depart=10 OR depart=20;*

La primera obtendrá empleados de los departamentos 10 y 20 que tengan sueldos mayores a 2000.

La segunda obtendrá empleados del departamento 10 con sueldo mayor a 2000 y empleados del departamento 20 con cualquier sueldo.

6.4 Operador de comparación de cadenas de caracteres (LIKE)

Este operador se utiliza para comparar cadenas de caracteres.

Permite utilizar caracteres comodines en las cadenas de comparación:

- **%** representa cualquier cadena de 0 o más caracteres
- _ representa un único carácter cualquiera

Nota

Hemos de tener en cuenta que las mayúsculas y minúsculas son significativas: 'm' no es lo mismo que 'M' y que las constantes alfanuméricas deben encerrarse siempre entre comillas simples.

Veamos algunos ejemplos:

```
-- Nombre de los polideportivos que están en una ciudad

-- cuyo nombre empieza por Z y tiene 8 caracteres

SELECT nombre
```

```
FROM polideportivos

WHERE ciudad LIKE 'Z_______';

-- Nombre de los polideportivos que están en una ciudad

-- cuyo nombre empieza por Z

SELECT nombre

FROM polideportivos

WHERE ciudad LIKE 'Z%';
```

6.5 Operadores IS NULL y IS NOT NULL

Se dice que una columna de una fila es NULL si está completamente vacía. Para comprobar si el valor de una columna es nulo empleamos la expresión: *columna* IS NULL.

Si queremos saber si el valor de una columna no es nulo, utilizamos: *columna* IS NOT NULL.

Cuando comparamos con valores nulos o no nulos no podemos utilizar los operadores de igualdad, mayor o menor.

Ejemplos:

```
-- Nombre y apellidos de los usuarios que no indicaron su fecha de
nacimiento

SELECT nombre, apellidos

FROM usuarios

WHERE fecha_nacimiento IS NULL;

-- Nombre y apellidos de los usuarios que indicaron su fecha de
nacimiento

SELECT nombre, apellidos

FROM usuarios

WHERE fecha_nacimiento IS NOT NULL;
```

6.6 Operador IN

El operador IN nos permite comprobar si una expresión pertenece o no (NOT) a un conjunto de valores, haciendo posible la realización de comparaciones múltiples. Su formato es:

<expresión>[NOT] **IN** (lista de valores separados por comas).

Ejemplos:

```
-- Nombre y extensión de los polideportivos de

-- Zaragoza, Huesca y Teruel

SELECT nombre, extension
```

```
FROM polideportivos

WHERE ciudad IN ('Zaragoza', 'Huesca', 'Teruel');

-- Nombre y extensión de los polideportivos que no son de

-- Zaragoza, Huesca y Teruel

SELECT nombre, extension

FROM polideportivos

WHERE ciudad NOT IN ('Zaragoza', 'Huesca', 'Teruel');
```

6.7 Operador BETWEEN

Este operador comprueba si un valor está comprendido o no dentro de un rango de valores, desde un valor inicial a un valor final. [NOT] BETWEEN valor_ini AND valor_fin

Ejemplos:

```
-- Nombre y apellidos de los usuarios que tienen un descuento

-- entre 10 y 20 %

SELECT nombre, apellidos

FROM usuarios

WHERE descuento BETWEEN 0.1 AND 0.2;

-- Nombre y apellidos de los usuarios que no tienen un descuento

-- entre 10 y 20 %

SELECT nombre, apellidos

FROM usuarios

WHERE descuento NOT BETWEEN 0.1 AND 0.2;
```

Nota

Un error relativamente frecuente consiste en utilizar expresiones del tipo:

1000 <= SALARIO <= 2000

Este tipo de expresiones es ilegal y provocará un error ya que al evaluar la primera parte de la expresión se sustituirá por un valor lógico de tipo *true/false* y este resultado no puede compararse con un valor numérico. La expresión correcta sería:

SALARIO >= 1000 AND SALARIO <= 2000

6.8 Operadores de concatenación

Para unir dos o más cadenas se utiliza el operador de concatenación +

Ejemplo:

```
SELECT 'Buenos' + ' días';
```

Daría como resultado 'Buenos días'.

7 Cláusula TOP.

La palabra clave "top" se emplea para obtener sólo una cantidad limitada de registros, los primeros n registros de una consulta.

Ejemplo:

```
-- Mostramos las 10 primeras filas de la tabla usuarios ordenadas por apellidos y
-- nombre
SELECT TOP 10 *
FROM usuarios
ORDER BY apellidos, nombre;
```

8 Funciones agregadas o de resumen.

Son funciones que permiten realizar operaciones sobre un conjunto de filas de una tabla, devolviendo un único resultado.

8.1 Función COUNT().

Devuelve el número de filas seleccionadas

```
-- Número de pistas

SELECT COUNT(*)

FROM pistas;

 -- Número de polideportivos en Zaragoza

SELECT COUNT(*)

FROM polideportivos

WHERE ciudad = 'Zaragoza';
```

8.2 Función SUM().

Devuelve la suma de todos los valores de una columna

```
-- Cuánto dinero costaría alquilar todas las pistas del

-- polideportivo cuyo id es 23

SELECT SUM(precio)

FROM pistas

WHERE id_polideportivo = 23;
```

8.3 Función MIN()

Devuelve el valor mínimo de una columna

```
-- Cuánto vale la pista más barata
```

```
SELECT MIN(precio)

FROM pistas;
```

8.4 Función MAX()

Devuelve el valor máximo de una columna

```
-- Cuánto vale la pista más cara

SELECT MAX(precio)

FROM pistas;
```

8.5 Función AVG()

Devuelve el valor medio de los valores de una columna

```
-- Valor medio de las pistas

SELECT AVG(precio)

FROM pistas;
```

Hay que tener en cuenta que, excepto la función `COUNT`, todas las demás devolverán el valor `NULL` si no hay columnas sobre las que puedan operar. La función `COUNT`, sin embargo, devolverá el valor 0 en ese caso.

9 Funciones escalares

Las funciones escalares toman un valor de entrada para devolver un resultado único a la consulta que la invoco.

Veamos algunas funciones escalares.

9.1 Función UPPER()

Convierte una cadena de caracteres a mayúsculas.

```
SELECT UPPER('SQL es un lenguaje de consulta estructurado');
```

9.2 Función LOWER()

Convierte una cadena de caracteres a minúsculas.

```
SELECT LOWER('SQL es un lenguaje de consulta estructurado');
```

9.3 Función DATALENGTH() o LEN()

Devuelve la longitud de una cadena de caracteres.

```
SELECT DATALENGTH('SQL es un lenguaje de consulta estructurado');
```

9.4 Función CONCAT()

Concatena dos cadenas de caracteres.

```
SELECT CONCAT('Buenos', ' días');
```

9.5 Función TRIM()

Elimina los espacios iniciales y finales de una cadena de caracteres.

```
SELECT TRIM('      ¡Curso de SQL!      ')
```

9.6 Función REPLACE()

Reemplaza un carácter por otro carácter nuevo en una cadena de caracteres determinada.

```
--Sustituye el carácter ‘a’ por el carácter ‘c’ en el texto.

SELECT REPLACE('ABC ABC ABC', 'a', 'c');
```

9.7 Función REVERSE()

Invierte una cadena de caracteres.

```
SELECT REVERSE('AGALAM');
```

9.8 Función SYSDATETIME()

Devuelve la fecha y la hora actual.

```
SELECT SYSDATETIME();
```

9.9 Función ROUND()

Devuelve un valor numérico, redondeado a la longitud o precisión especificadas.

```
--Redondea el número a dos decimales

SELECT ROUND(235.415, 2)
```

9.10 Funciones de tipo fecha

Podremos utilizar las funciones DATEADD y DATEDIFF, si deseamos sumar o restar fechas.

Ejemplo

```
-- Muestra los usuarios que hayan nacido en el 2013 ordenados por fecha de
nacimiento

SELECT *
FROM usuarios
WHERE fecha_nacimiento >= '2013-01-01' AND fecha_nacimiento < DATEADD(YEAR, 1,
'2013-01-01')
ORDER BY fecha_nacimiento;

-- Muestra el resultado de sumar 12 meses a la fecha indicada
SELECT DATEADD(MONTH, 12, '2013-01-01');

-- Muestra la diferencia en días entre las dos fechas indicadas
SELECT DATEDIFF(DAY, '2022-01-01', '2022-02-23');
```

10 Cláusula GROUP BY

Sirve como su propio nombre indica para hacer agrupaciones de datos.

Las funciones de agregado, por sí solas, devuelven un valor de resumen aplicado a todas las filas de una tabla. Podemos generar valores de resumen agrupando los registros por un campo determinado.

Por ejemplo, si deseamos conocer la cantidad de libros de cada editorial, realizaremos la siguiente consulta:

SELECT editorial, count(*)

FROM libros

GROUP BY editorial;

Ejemplos:

```sql
-- Número de polideportivos hay en cada ciudad

SELECT ciudad, COUNT(*) AS cantidad

FROM polideportivos

GROUP BY ciudad;

-- Número de usuarios en cada ciudad

SELECT ciudad, COUNT(*) AS cantidad

FROM usuarios

GROUP BY ciudad;
```

11 Cláusula HAVING

Si deseamos aplicar condiciones a un agrupamiento, en vez de usar la cláusula WHERE, debemos usar la cláusula HAVING.

Ejemplo:

```sql
-- Número de polideportivos hay en cada ciudad, solamente de aquellas

-- ciudades donde hay más de 10

-- Si usamos la cláusula WHERE para especificar una condición al

-- agrupamiento nos dará un error. Por tanto, habrá que utilizar la

-- cláusula HAVING.

SELECT ciudad, COUNT(*) AS cantidad

FROM polideportivos

WHERE COUNT(*) > 10

GROUP BY ciudad;

-- Usamos la cláusula HAVING.

SELECT ciudad, COUNT(*) AS cantidad

FROM polideportivos

GROUP BY ciudad

HAVING COUNT(*) > 10;
```

12 Consultas multitabla

A veces una consulta necesita columnas de varias tablas, en este caso el formato es:

SELECT columna1, columna2,…

FROM tabla1, tabla2,…

Podemos unir tantas tablas como deseemos (aunque la experiencia aconseja que no sean más de 8), siempre y cuando cada una de las tablas tengan un campo en común, que será el que se utiliza para realizar la unión.

En la cláusula SELECT podemos usar columnas de todas las tablas.

Si hay columnas con el mismo nombre en las distintas tablas de FROM, se debe especificar: Nombretabla.columna

El criterio que se siga para combinar las tablas debe especificarse en la cláusula WHERE, si no se especifica ningún criterio, el resultado de la combinación será un producto cartesiano, que emparejará todas las filas de una tabla con todas las filas de la otra.

Ejemplo:

```
-- Mostramos la información de las pistas con sus reservas
SELECT pistas.id 'pistas_id', codigo, tipo, reservas.id 'reservas_id',
fecha_reserva, fecha_uso
FROM pistas, reservas
WHERE pistas.id = reservas.id;

-- Realizamos el producto cartesiano entre las tablas pistas y reservas
-- Cada fila de la tabla pistas se combinará con cada una de las filas de la
-- tabla reservas. Por tanto, la consulta devolverá:
-- número de filas de la tabla pistas x número de filas de la tabla reservas
-- 190 x 200 = 38.000 filas
SELECT pistas.id 'pistas_id', codigo, tipo, reservas.id 'reservas_id',
fecha_reserva, fecha_uso
FROM pistas, reservas;
```